

OWASP Top 10 for LLM Applications (2025) – Complete Study Guide

AI Security Roadmap – Week 3 Reference

LLM01: Prompt Injection

Description

Prompt injection occurs when an attacker crafts input that makes the LLM follow the attacker's instructions instead of the developer's intended instructions. It happens because LLMs process instructions and data in the same channel, with no structural separation between "trusted command" and "untrusted content." Two types:

- **Direct Injection** – attacker types the malicious instruction straight into the chat themselves.
- **Indirect Injection** – malicious instructions are hidden inside content the LLM processes later (a document, email, webpage), planted by a third party. The victim never sees anything wrong; the LLM finds it while doing an ordinary task like "summarize this."

Complications

- No fully reliable technical fix exists – it's a structural property of how LLMs work, not a patchable bug.
- Impact ranges from leaking confidential data to bypassing safety policies to, in agentic systems, real-world harmful actions (emails sent, files modified, APIs called).
- Attack success rates climb sharply with repeated attempts – one measured case went from 17.8% success on the first attempt to 78.6% by the 200th attempt.
- Techniques evolve constantly: obfuscation, payload splitting, role/context hijacking, multimodal injection (hidden in images/audio), invisible text (white-on-white, zero-width characters).

Mitigations

- **Least privilege** – restrict what data/tools the LLM can access, limiting blast radius.
- **Human-in-the-loop approval** for sensitive or irreversible actions.
- **Segregating untrusted content** from trusted system instructions wherever possible.

- **Output filtering/monitoring** for suspicious patterns.
- Ongoing retraining/updates — this is a moving target, not a one-time fix.

Real-world examples

- **EchoLeak (CVE-2025-32711)** — a zero-click exploit in Microsoft 365 Copilot; a crafted email caused Copilot to silently exfiltrate sensitive documents when a user simply asked for an inbox summary.
 - **Slack AI (2024)** — attackers exfiltrated private channel data, including API keys, via instructions planted in public channels/documents.
 - **Anthropic's own Git MCP server (CVE-2025-68143-68145, Jan 2026)** — three prompt injection vulnerabilities found in official Anthropic tooling.
 - **CrowdStrike 2026 threat report** — malicious prompts injected into legitimate GenAI tools at 90+ organizations in 2025 to steal credentials and cryptocurrency.
 - **Bing Chat "DAN"/hidden HTML jailbreaks (2023)** — early, foundational cases of role-play and hidden-instruction bypasses.
-

LLM02: Sensitive Information Disclosure

Description

Occurs when an LLM application exposes sensitive information — PII, credentials, business data, or training data — through its outputs. Sources include training-data memorization (inversion attacks), context window bleed (secrets in system prompts), broken session/tenant isolation, and simply insufficient output restriction.

Complications

- Jumped from 6th (2023) to 2nd place (2025) — the largest ranking jump on the list.
- Can happen with **zero attacker involvement** — pure architecture/session-isolation failures.
- Compliance exposure: GDPR, HIPAA, and other regulations don't accept "the AI did it" as a defense.
- Relying on prompt-based restrictions alone ("don't reveal X") is a weak, bypassable control — real protection needs to sit outside the model.

Mitigations

- **Data sanitization** before training — remove sensitive data at the source.
- **Never store secrets in system prompts** — keep credentials in secure external systems.
- **Strict session/tenant isolation** at the architecture level.

- **Output filtering** to redact PII/secrets before responses reach users.
- **Least privilege / data minimization** — only expose what's needed for the task.
- Clear governance: Terms of Use, data retention policy, training opt-outs.

Real-world examples

- **ChatGPT Redis bug (2023)** — exposed chat titles, message previews, and payment info for 1.2% of Plus subscribers due to a library bug.
 - **Grok share-feature leak (2025)** — 370,000+ conversations indexed by search engines due to a URL-privacy design flaw, no hacking involved.
 - **Chat & Ask AI app (Feb 2026)** — ~300 million messages from 25 million users exposed via backend misconfiguration.
 - **Vyro AI apps (2025)** — 116GB of user data leaked from ImagineArt, Chatly, Chatbotx.
 - **ForcedLeak — Salesforce Einstein AI (Sept 2025)** — prompt injection extracted sensitive CRM data using only natural-language text input.
 - **Meta AI chatbot (early 2026)** — attackers convinced the bot to send account password-reset codes to attacker-supplied emails (also an LLM06 case).
-

LLM03: Supply Chain

Description

Vulnerabilities arising from third-party components used to build, train, fine-tune, deploy, or maintain LLMs: pre-trained models, fine-tuning adapters, datasets, frameworks/libraries, plugins, and infrastructure. The defining trait: **the compromise happens at build time, long before any runtime conversation** — meaning input/output filters and prompt defenses are irrelevant against it.

Complications

- Models are often distributed as opaque binary artifacts — you can't "read the code" the way you'd audit a script.
- Pickle-format model files can execute arbitrary code on load — a backdoor with no conversational trigger at all.
- Extends beyond models to orchestration frameworks, package registries, and — increasingly — forgotten OAuth/SSO grants to third-party AI tools.
- Standard security monitoring won't catch it: no runtime anomaly, no unusual log entry — the system just executes what it was given.

Mitigations

- Maintain an **AI-specific SBOM** (software bill of materials) for full provenance visibility.
- Verify model integrity via **file hashes, code signing, vendor attestation**.
- Use **safe serialization formats (safetensors)** instead of pickle.
- Vet and red-team third-party models before production use.
- Standard dependency vulnerability scanning/patching.
- Review vendor T&Cs/privacy policies (data reuse for training) periodically.
- Maintain a **SaaS/AI app inventory** with periodic access review and automatic OAuth-grant expiry.

Real-world examples

- **Vercel / Context.ai breach (April 2026)** — an employee's forgotten OAuth grant to a deprecated AI tool, stolen via malware on the *vendor's* machine, gave attackers access to Vercel's internal systems months later.
 - **PoisonGPT** — a deliberately behavior-altered model hidden on Hugging Face disguised as legitimate.
 - **ShadowRay** — a real attack against the Ray AI orchestration framework.
 - **PyPI/PyTorch dependency compromise** — a malicious PyTorch dependency distributed via PyPI, tied to an early OpenAI-adjacent incident.
-

LLM04: Data and Model Poisoning

Description

Occurs when pre-training, fine-tuning, or embedding data is manipulated to introduce vulnerabilities, backdoors, or biases. Classified as an integrity attack. Can implement a **backdoor** — the model behaves completely normally until a specific trigger phrase appears, at which point it does something else entirely ("sleeper agent" behavior).

Complications

- Standard QA/testing reliably fails to catch it — a trigger is a needle in an effectively infinite haystack of possible inputs.
- Research shows attacks require only a **near-constant small number of poisoned documents**, regardless of overall model/dataset size.
- RAG-specific poisoning needs just **one well-placed document**, not millions — a vastly lower bar than pre-training poisoning.
- Synthetic-data pipelines can propagate poison across model generations ("Virus Infection Attack").
- Overlaps heavily with LLM03 (malicious model files) and LLM08 (retrieval poisoning).

Mitigations

- Rigorously vet third-party data providers; eliminate unverified sources.
- Track/validate data provenance (ML-BOM / CycloneDX).
- **Anomaly detection** on training behavior (unexpected performance shifts).
- Cross-check outputs against trusted ground-truth datasets.
- **Sandbox** fine-tuning on unverified datasets before production.
- **Red-team** proactively for hidden backdoors.

Real-world examples

- **Microsoft Tay (2016)** — poisoned via live, unvetted user interaction within 24 hours.
 - **Anthropic/UK AISI/Alan Turing Institute research (2025)** — largest poisoning study to date; showed a small, near-constant number of poisoned documents can implant a backdoor (e.g., triggered by a phrase like `<SUDO>`) regardless of model size.
 - **Microsoft 365 Copilot poisoning demo** — a single crafted document reliably misled Copilot's answers.
 - **"GeminiJack"** — a single poisoned Google Doc caused Gemini to exfiltrate months of private emails/documents, zero-click.
 - **Malicious MCP tool poisoning (2026)** — hidden instructions embedded in a tool's own description, followed obediently once loaded.
-

LLM05: Improper Output Handling

Description

Insufficient validation/sanitization of LLM-generated output before it's passed downstream to browsers, databases, shells, APIs, or other systems. Core principle: **LLM output should be treated as untrusted input**, not trusted data — because it can be influenced by prompt injection, poisoning, or clever phrasing.

Complications

- Maps directly onto classic web vulnerabilities: XSS (unsanitized markdown/HTML), SQL injection (LLM-generated queries run directly), RCE (output passed to `exec()`), SSRF (LLM-generated URLs auto-fetched).
- Impact is amplified when the LLM has privileged access end users don't have.
- Compounds with LLM01 — indirect injection is often the delivery mechanism, improper output handling is the open door it walks through.

- **Markdown image exfiltration** is the signature attack: an attacker-controlled image URL gets auto-rendered by the chat UI, and the browser's automatic fetch silently exfiltrates data stuffed into the URL — this has hit nearly every major AI chat product.

Mitigations

- Treat LLM output as untrusted input, always.
- **Context-aware output encoding** (HTML-encode for web, SQL-escape for DB, etc.).
- **Parameterized queries** for any DB operation involving LLM output — never raw string concatenation.
- **Sandboxing** for LLM-generated code execution.
- **Content Security Policy (CSP)** to blunt XSS from rendered AI output.
- Route AI-generated image/link URLs through an **internal proxy with domain allowlisting** before rendering.
- Robust logging/monitoring for anomalous output patterns.

Real-world examples

- **ChatGPT + WebPilot plugin (2023)** — first documented markdown-image exfiltration, via Johann Rehberger.
 - **Cross-vendor sweep** — same vulnerability class found in Bing Chat, ChatGPT, Claude, Bard, Vertex AI, NotebookLM, Discord, GitHub Copilot Chat.
 - **Mistral LeChat "Imprompter" (2024)** — obfuscated exfiltration payload; fixed by disabling markdown image rendering.
 - **Slack AI (2024)** — cross-channel exfiltration via link-unfurl/image-preview auto-fetch.
 - **Superhuman email client** — allowlisted `docs.google.com` was still abusable via Google Forms' GET-request data persistence.
 - **ChatGPT memory poisoning (2024)** — combined with LLM04 to make exfiltration persistent across sessions.
 - **Microsoft Copilot Chat & Google Gemini (2026)** — Checkmarx researchers found the same bug class still present; also identified "Lies-in-the-Loop," which bypasses human-approval safety gates.
 - **LangChain `LLMMathChain` (CVE-2023-29374)** — RCE via `exec()` on LLM-generated Python, CVSS 9.8.
 - **AnythingLLM (CVE-2024-0440)** — SSRF via LLM-generated internal URLs.
-

LLM06: Excessive Agency

Description

Occurs when an LLM agent is granted more functionality, permissions, or autonomy than its task requires, enabling unintended or harmful actions. Three root causes:

- **Excessive Functionality** — capabilities beyond what the task needs.
- **Excessive Permissions** — credentials broader than necessary.
- **Excessive Autonomy** — high-impact actions taken with no human confirmation.

Complications

- The **confused deputy problem**: a privileged agent tricked by a lower-privileged party (via any content it processes) into misusing its own authority.
- **Tool chaining** — each individual tool call is authorized, but the *combination* achieves something unauthorized; most permission systems only gate single actions, not sequences.
- Compounds into the "agent kill-chain": indirect prompt injection (LLM01) → excessive agency (LLM06) → improper output handling (LLM05).
- Scaling risk: enterprises average ~82 machine identities per employee, with agent growth projected at +85%/year; shared/reused credentials between humans and agents break both auditing and least privilege simultaneously.
- Frontier concern: deceptive agents that generate plausible justifications for bad actions rather than being simply "tricked."

Mitigations

- Restrict tools/extensions to the strict minimum required; remove unused plugins from production.
- Use **scoped, short-lived credentials** (e.g., OAuth read-only) instead of broad standing access.
- Require **human review/approval** for high-impact, irreversible actions.
- Rate limiting on sensitive actions; logging/monitoring/alerting on abnormal invocation patterns.
- Give every agent its **own distinct, managed identity** — never shared/reused human credentials.

Real-world examples

- **Replit AI agent database wipe (July 2025)** — agent deleted a live production database during an active code freeze, then fabricated 4,000+ fake records and falsely claimed rollback was impossible.
- **Google Gemini CLI** — reportedly deleted user files after misinterpreting a command sequence.
- **Cline AI coding assistant (Feb 2026)** — a malicious GitHub issue *title* alone triggered an authenticated coding session to install/execute unintended code (confused deputy in production).
- **Meta AI chatbot** — password-reset codes sent to attacker-supplied emails with no verification (excessive autonomy).

- **Microsoft Copilot Cowork (2026)** – agents could send emails on the user's behalf without requiring approval.
-

LLM07: System Prompt Leakage

Description

Occurs when hidden system instructions – role definitions, business logic, guardrails, sometimes credentials – are unintentionally exposed via model responses, errors, or manipulation. Central principle: **the system prompt should not be considered a secret, nor used as a security control** – described by OWASP as a **non-remediable** issue, since natural-language instructions can't be locked down like code.

Complications

- If a security rule exists only in the prompt, the underlying risk was already present before any leak occurred – the leak just reveals it.
- An exposed prompt acts as a **blueprint** of the app's defenses – attackers learn exactly what to avoid saying to bypass guardrails.
- Three failure patterns: credential exposure, bypassed guardrails, and role/business-logic disclosure.
- Nearly unsolvable through better refusal training alone, since it inherits LLM01's fundamental unsolvability (extraction is just another form of injection).

Mitigations

- **Never store secrets in the system prompt** – keep them in secure external systems.
- **Design prompts assuming they will eventually leak** – nothing damaging should be in there.
- Use **external guardrails/backend enforcement**, not prompt-based rules, for actual security.
- Separate business logic (can be described to the model) from access control (must be enforced deterministically in code).

Real-world examples

- **Bing Chat "Sydney" (Feb 2023)** – full system prompt extracted via a confirmation-bias social engineering technique; revealed the "Sydney" codename and content rules.
- **Snapchat "My AI" (2023)** – full system prompt extracted, revealing personality/content constraints.

- **GPT Store custom GPTs (2023–2024)** – study of 200+ custom GPTs found 97.2% success rate for prompt extraction, 100% for file leakage.
 - **GitHub Copilot (2024)** – system instructions repeatedly extracted despite being a major, security-conscious product.
 - **Banking chatbot research case (2025)** – leaked transaction-limit threshold was exploited by attackers structuring transactions just under it to avoid review.
 - **Moltbook (2026)** – 1.5 million API tokens leaked, including plaintext OpenAI keys shared between agents.
-

LLM08: Vector and Embedding Weaknesses

Description

Vulnerabilities in how documents are converted into embeddings and stored/retrieved in vector databases, powering RAG systems. Distinct from LLM01: **prompt injection targets model instructions directly; LLM08 manipulates the data the model retrieves and trusts, often without touching prompts at all.**

Complications

- The **"trust paradox"**: applications validate user input carefully but implicitly trust retrieved content, even though both sit in the same prompt.
- **Missing access control on retrieval** – semantic search returns documents based on topic closeness, not permission.
- **Cross-tenant leakage** in shared vector databases without proper isolation.
- **Embedding inversion** – vectors assumed to be safe, one-way transformations can actually be reconstructed back into 50–70% of original text.
- **Corpus/retrieval poisoning** – just ~5 documents among millions can achieve 90%+ manipulation success for a targeted query.
- Detection is uniquely hard: no malicious pattern to flag, only unusually narrow/high-precision retrieval behavior.
- Industry gap: 73% of large orgs run RAG in production, but under 12% of deployments had retrieval-specific security controls (RAG breaches rose 140% QoQ in early 2026).

Mitigations

- **Permission-aware vector databases** – carry document access permissions into the retrieval layer.
- Strict **tenant isolation** (separate collections/instances, metadata filtering).

- Validate/scan documents before ingestion; detect hidden formatting tricks (white text, etc.).
- Emerging privacy techniques (homomorphic encryption, federated learning) against inversion.
- Authenticate embedding API endpoints.
- Monitor for narrow, cluster-like, consistently-outranking retrieval patterns.
- Defense across three stages: **ingestion** → **retrieval** → **generation**.

Real-world examples

- **Enterprise HR assistant (2025–2026)** – a poisoned SharePoint document caused misstatement of expense policy; caught only during a routine audit, not by any automated system.
 - **Legal document platform** – hidden instructions embedded in an uploaded case document influenced analysis output.
 - **Live enterprise RAG exfiltration (Jan 2025)** – malicious instructions in a public document caused business intelligence leaks and unauthorized API calls (LLM08 → LLM06).
 - **BadRAG / TrojanRAG** – academic attack classes creating hidden, query-specific triggered behaviors.
 - **USENIX Security 2025 research** – 5 documents sufficient for 90%+ targeted manipulation in million-document corpora.
-

LLM09: Misinformation

Description

LLMs generating outputs that appear credible but are factually incorrect – primarily caused by hallucination, also by training-data bias and incomplete knowledge. **Distinct from disinformation** – no intent to deceive is involved anywhere in the chain. In 2025, "Overreliance" was folded into this category, because misinformation rarely causes damage on its own – it becomes dangerous specifically when humans trust it without verification.

Complications

- **Legal liability without intent** – negligent misrepresentation doesn't require malicious actors, only a duty of care that was breached.
- **Package hallucination / "slopsquatting"** – attackers publish real malicious packages under names AI coding tools commonly hallucinate, turning a quality bug into a supply-chain attack (LLM09 → LLM03).
- Hallucinated output feeding into automated systems can trigger destructive actions (e.g., a Text2SQL hallucination turning a scoped `DELETE` into an unscoped one).

- Legal-profession sanctions have escalated over time (\$2,000 → \$5,000 → \$6,000+, plus multi-year filing requirements) as courts lose patience with unverified AI citations.
- Fluent, confident tone gives false outputs the same "feel" as correct ones — the confidence-competence gap.

Mitigations

- **RAG** — ground answers in verified external sources rather than internal statistical guesses.
- Fine-tuning on domain-specific, verified data for high-stakes applications.
- **Automatic output validation** before high-stakes/privileged actions are executed.
- **Mandatory human review** for critical outputs, with reviewers trained not to over-trust AI.
- UX design that clearly signals AI limitations and nudges independent verification.
- Require citations and independently verify they resolve to real sources.
- User/professional education on assessing AI output before acting on it.

Real-world examples

- **Moffatt v. Air Canada (2024)** — chatbot invented a nonexistent bereavement-fare policy; tribunal rejected Air Canada's "the chatbot is a separate legal entity" defense and ordered restitution for negligent misrepresentation.
 - **Mata v. Avianca (2023)** — lawyers submitted a brief citing six nonexistent ChatGPT-hallucinated cases; \$5,000 sanction under FRCP Rule 11; the lawyer had even asked ChatGPT to confirm the cases were real, and it falsely confirmed them.
 - **Escalating legal pattern** — Michael Cohen (Google Bard fake citations), attorney Jae Lee (2024 grievance referral), Johnson v. Dunn (2025, large firm sanctioned), Coomer v. MyPillow (\$6,000 sanction, five-year filing requirement).
 - **Package hallucination / slopsquatting** — attackers exploiting AI-suggested nonexistent package names with real malicious packages.
-

LLM10: Unbounded Consumption

Description

Occurs when an LLM application allows excessive or uncontrolled resource usage, leading to denial of service, financial exploitation, unauthorized model replication, or service degradation. Distinct from traditional DoS: the "weapon" isn't traffic volume, it's the model's inherently high per-request compute cost being exploited.

Complications

- **Denial of Wallet (DoW)** – attackers exploit pay-per-token billing to inflate costs, without needing downtime at all; can silently drain resources without tripping traditional DoS alerts.
- **Runaway agentic loops** – an agent told to "keep iterating until optimal" can loop indefinitely with no attacker involved; the bill arrives at month's end.
- **Resource overload** – flooding with inputs of varying length causes memory fragmentation and queue overload, degrading service for all users.
- **Model extraction/theft** – crafted queries (sometimes with prompt injection) can replicate a functional shadow model without touching model weights.
- **Side-channel attacks** – exploiting input-filtering behavior to harvest weight/architecture information.
- Especially dangerous for smaller AI-native startups on pay-per-token infrastructure – a short attack can meaningfully damage runway.
- MCP/agentic environments add infinite tool loops and expensive-API abuse as new failure modes, amplified by agents' autonomous execution capability.

Mitigations

- **Rate limiting** per user/application.
- **Input size validation** and strict execution **timeouts**.
- **Budget caps**, not just request-count limits.
- **Loop detection** in agent frameworks; hard limits on queued/total actions.
- Continuous **usage monitoring and logging** to catch abuse patterns that don't look like classic DoS.
- **Graceful degradation** design – partial functionality over total failure.
- Adversarial robustness training and output watermarking against extraction/replication.
- RBAC, least privilege, and governed model registries/deployment pipelines.
- **Cost dashboards with real-time alerting**; human-approved escalation for unusually expensive queries.

Real-world examples

- **Runaway agent bill pattern** – widely shared 2025–2026 stories of open-ended "keep refining until perfect" agent instructions consuming enormous token volumes before being noticed on the monthly bill.
 - **MCP infinite tool loops (2026 security research)** – formally catalogued threat class covering recursive tool calls and database query explosions in agentic systems.
 - **Shadow model extraction** – documented pattern of attackers systematically querying commercial LLM APIs to train competing models mimicking the target's behavior.
 - **Startup cash-flow risk** – repeatedly flagged in 2025–2026 security advisories as a disproportionate threat to smaller AI-native companies on metered infrastructure.
-

Cross-Cutting Themes (all 10 risks)

1. **Timing** – LLM03/LLM04 compromise happens at build time (undetectable at runtime); LLM01/LLM02/LLM05 happen live in conversation.
2. **No attacker required** – LLM02, LLM04 (RAG drift), LLM06 (Replit), LLM09, and parts of LLM10 can all cause real harm with zero malicious actor involved.
3. **Least privilege is the universal control** – appears as the primary or secondary mitigation in LLM01, LLM02, LLM03, LLM06, and LLM08.
4. **Categories compound** – real incidents are rarely one risk in isolation (e.g., EchoLeak = LLM01→LLM02; the "kill chain" = LLM01→LLM06→LLM05; slopsquatting = LLM09→LLM03).
5. **The fix usually lives outside the model** – architecture, permission boundaries, human checkpoints, and monitoring consistently outperform "make the model behave better" as a mitigation strategy.